

NovaPharma Iberia — Step 0 Review: The Excel Model As-Is

Digital transformation project · Excel → MySQL / Python / Power BI / Anaplan · Author: Nix · 19 September 2026

Objective A — Understand what every sheet does

The legacy workbook *NovaPharma_Commercial_Model.xlsx* has 17 sheets. Each one was reviewed and classified: what it is, what it contains (data, formulas or macro support) and where it will live in the target architecture. The full dictionary is in section D.

Reading the workbook this way reveals that it already contains five distinct layers — documentation and configuration, data, transformation, reporting and operations — mixed in a single file. Those five layers are exactly the five components of the target architecture. The transformation does not invent a new structure: it separates one that was already there.

Objective B — The sheets that form the data model

Eight sheets hold data and become the warehouse: four dimensions (Calendar, Products, Customers, Territories) and four facts (Sales_Raw, Budget, Forecast, Market). Sales_Raw is the source of truth for actuals; Sales_Enriched is derived from it (Raw plus lookups) and is therefore not migrated as data. Their modelling is documented in the Step 1 and Step 2 reviews.

Objective C — The other nine sheets and where they will live

Layer	Sheets	Role in Excel	Where it lives after the migration
Documentation & configuration	README, Parameters	Manual of the file; input cells (CurrentMonth, LatestForecast, VarThreshold) that drive every formula	README → 'About' page in Power BI + technical README of the repository. Parameters → Power BI slicers/parameters; CurrentMonth → dim_calendar.is_closed maintained by the pipeline
Transformation	Sales_Enriched	Hand-made JOIN: Raw + 11 lookup/derived columns (~530k formulas)	SQL view v_sales_enriched
Reporting	Consolidation, BvA, KPI_Dashboard	GROUP BY grids (SUMIFS family), Actual vs Budget vs Forecast, management KPIs and charts	Power BI (DAX measures + visuals) on top of shared SQL views (v_bva)
Operations	Data_Quality, Import_Staging, Macro_Log	Quality checks (formulas), CSV landing zone, audit trail written by the macros	Python pipeline: validation + quarantine table, staging tables stg_*, logging / etl_log; MySQL PK/FK constraints

Nothing is lost: every function moves to the layer built for it. What disappears is the manual work — pressing the macro button, changing CurrentMonth by hand, fixing bad rows directly in Sales_Raw — and with it the single owner who knew how the file worked.

Extra — Simplifying the BI layer

The Excel reporting sheets contain roughly 20 distinct visuals. For this project the Power BI report is reduced to **10 visuals on 2 pages**; the rest is recorded as a backlog the client can develop later if genuinely needed.

Page	Visuals	Replaces
1 · Overview	4 KPI cards (Net Sales YTD, Var vs Budget %, GM %, Market share) · 12-month line Actual / Budget / Forecast · Net Sales by brand bar chart with drill-down to SKU · Region table	KPI_Dashboard tiles, trend, top-5, brand and region tables; Consolidation brand × region × month (via drill-down + month filter)
2 · Budget vs Actual	Brand × territory matrix with status colours · Version slicer · Threshold parameter	BvA sheet; Parameters
Backlog (not built)	TA × channel matrix, brand statistics table, 8 secondary KPI cards, top-5 table	Consolidation blocks 3–4, remaining KPI tiles

D — Sheet dictionary (17 sheets)

#	Sheet	What it is	Contains	Destination
1	README	Documentation for whoever opens the file: purpose, sheet map, monthly process, macros, limitations	Text	Project documentation / Power BI About page
2	Parameters	Model drivers: CurrentMonth, LatestForecast, VarThreshold + derived helpers; named ranges; blue/yellow input cells	Inputs + small formulas	Power BI slicers/parameters; dim_calendar.is_closed
3	Calendar	Time dimension, one row per month	Data	dim_calendar
4	Products	Product dimension, one row per SKU (brand, TA, prices, launch/LoE dates)	Data	dim_product + dim_brand
5	Customers	Customer dimension (channel, territory, city)	Data	dim_customer
6	Territories	Territory dimension (region, country, sales rep)	Data	dim_territory
7	Sales_Raw	ERP transaction extract as received: 48,163 rows, 8 columns; carries the real data-quality issues	Data	fact_sales (after Python cleaning)
8	Sales_Enriched	Sales_Raw + 11 lookup/derived columns (VLOOKUP, INDEX/MATCH, Month_Key, Net Sales, COGS, GM, Dup_Flag); rebuilt by macro	~530k formulas	SQL view v_sales_enriched
9	Budget	Annual sales plan by version × month × SKU × territory; frozen once approved	Data	fact_plan
10	Forecast	Quarterly re-photograph of the plan: actuals to cut-off + estimates	Data	fact_plan
11	Market	IQVIA market data by month × brand × territory; used for market share	Data	fact_market
12	Consolidation	Aggregation grids: Net Sales & Units by brand × region × month; TA × channel YTD; brand statistics	SUMIFS / COUNTIFS / AVERAGEIFS / SUMPRODUCT	SQL GROUP BY views; Power BI matrices (partly backlog)
13	BvA	Budget vs Actual vs Forecast YTD by brand × territory: variances, run-rate, status flags	SUMIFS + conditional formatting	SQL view v_bva; Power BI page 2
14	KPI_Dashboard	Management view: KPI tiles, brand/region tables, top-5, 12-month trend, two charts	Formulas + charts	Power BI page 1
15	Data_Quality	11 live checks (duplicates, unknown codes, casing, spaces, returns, reconciliation) with OK/CHECK status; read by macro ValidateData	Formulas	Python validation + quarantine; MySQL constraints
16	Import_Staging	Empty landing zone with fixed headers; ImportMonthlyFile pastes, validates and appends the monthly CSV to Sales_Raw	Empty by design	MySQL staging tables stg_*
17	Macro_Log	Audit trail written by every macro (timestamp, macro, user, message); never edited by hand	Data written by macros	Python logging / etl_log table

Highlighted rows = the eight data sheets that form the model. Macros (VBA): ImportMonthlyFile, RefreshConsolidation, ValidateData, SnapshotForecastVersion, ExportToCSV — a macro is a VBA script that automates in Excel a sequence of actions otherwise done by hand; the pipeline replaces them with scheduled Python jobs.